Ariana Campbell

CIS 345

Professor Ramos

5/13/2026

Final Project Report: An xv6 Lottery Scheduler

In this project, I modified the xv6 operating system to implement a lottery scheduler. I added new system calls, updated the scheduler logic, tracked process statistics, and tested the scheduler using processes with different ticket values.

Screenshots:

```
xv6 kernel is booting

init: starting sh
$ lotterytest
PID     Tickets Ticks
4       30      106
5       20      79
6       10      36
$
```

1. **Lottery Scheduler Output:** This shows the final output from the lottery test program running in xv6. The process with 30 tickets received the most CPU time slices, while the process with 10 tickets received the least. This proves the lottery scheduler is working correctly.

```
enum procstate { UNUSED, USED, S

// Per-process state
struct proc {
int tickets;
int ticks;
  struct spinlock lock;
```

2. **Added Tickets and Ticks:** This shows the new tickets and ticks fields added to the struct proc structure in proc.h. The tickets variable stores how many lottery tickets a process has, while the ticks variable tracks how many times the process was scheduled to run.

```
p->tickets = 1;
p->ticks = 0;
```

3. **Default Ticket Initialization:** This shows the default initialization for new processes inside allocproc(). Every process starts with 1 ticket and 0 ticks before it begins running.

```
np->tickets = p->tickets;
np->ticks = 0;
```

4. **Child Process Ticket Inheritance:** This screenshot shows the changes made to fork() so child processes inherit the same number of tickets as their parent process. The child's tick count is also reset to 0 when it is created.

```
uint64
sys_pause(void)
{
  int n;
  uint ticks0;

  argint(0, &n);
  if(n < 0)
    n = 0;
  acquire(&tickslock);
  ticks0 = ticks;
  while(ticks - ticks0 < n){
    if(killed(myproc())){
      release(&tickslock);
      return -1;
    }
    sleep(&ticks, &tickslock);
  }
  release(&tickslock);
  return 0;
}
```

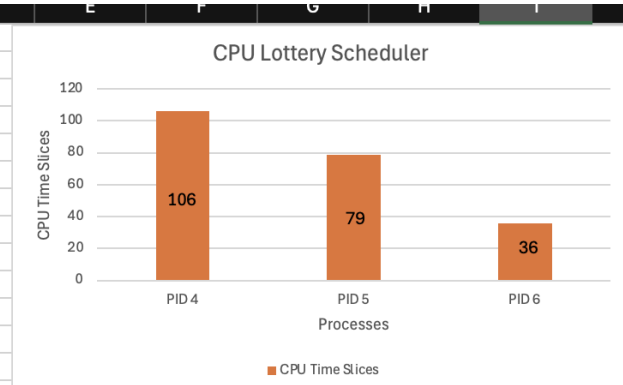
5. **sys_pause() System Call:** This shows the implementation of the sys_pause() system call in sysproc.c. The function pauses a process for a specified number of clock ticks by using the system timer and sleep() function. I used this in the lotterytest program to give the scheduler enough time to run the processes.

```c
uint64
sys_getpinfo(void)
{
  uint64 addr;
  struct pstat ps;
  struct proc *p;
  int i = 0;

  argaddr(0, &addr);

  if(addr == 0)
   return -1;

for(p = proc; p < &proc[NPROC]; p++){
  acquire(&p->lock);
  ps.inuse[i] = (p->state != UNUSED);
  ps.tickets[i] = p->tickets;
  ps.pid[i] = p->pid;
  ps.ticks[i] = p->ticks;

  release(&p->lock);

  i++;
}

if(copyout(myproc()->pagetable, addr, (char *)&ps, sizeof(ps)) < 0)
  return -1;

return 0;
}
```

6. **sys_getpinfo() System Call:** This shows the sys_getpinfo. The function goes through the process table and stores information like process ID, tickets, and ticks inside the pstat structure.

Graph

Process	Tickets	CPU Time Slices
PID 4	30	106
PID 5	20	79
PID 6	10	36



This graph shows how the lottery scheduler distributed CPU time between the three processes. PID 4 had the most tickets (30), so it was selected the most and ended up with 106 CPU time slices. PID 5 had 20 tickets and received 79 slices, while PID 6 had the fewest tickets (10) and only received 36 slices. Overall, the results show that the scheduler worked correctly because the processes with more tickets got more CPU time.

Overall, the lottery scheduler worked as expected. Processes with more tickets were selected more often and received more CPU time slices compared to processes with fewer tickets.